WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

The Arithmetic Logic Unit

Lecture 18: shifts and rotations, and selecting the operation

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 Shifts and rotations

02 Putting the ALU together

03 Summary

01

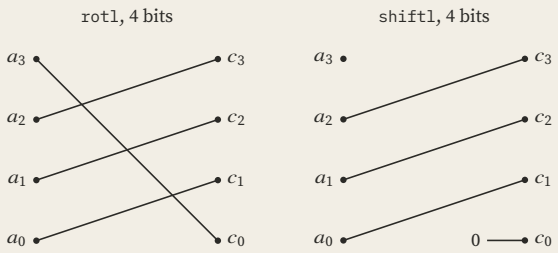
Shifts and rotations

Moving bits

a	a_7	a_6	a_5	a_4	a_3	a_2	a_1	a_0
`rotl`	a_6	a_5	a_4	a_3	a_2	a_1	a_0	a_7
`rotr`	a_0	a_7	a_6	a_5	a_4	a_3	a_2	a_1
`shiftl`	a_6	a_5	a_4	a_3	a_2	a_1	a_0	0
`shiftr`	0	a_7	a_6	a_5	a_4	a_3	a_2	a_1

- A *rotation* moves every bit one place; the bit that falls off one end comes back in at the other.
- A *shift* moves every bit one place too, but the bit that falls off is lost, and a 0 comes in at the other end.
- The DX8 has all four, on regA `(Unit 15)`.

No gates needed



- Output bit c_i is just input bit a_{i-1}: a shift or a rotation is a pattern of wires, with no gates at all.
- `shiftl` also needs a constant 0, and leaves a_3 unconnected.
- In Logisim-evolution the book builds them with splitters, which regroup the wires of a bus.

Shifts are arithmetic too

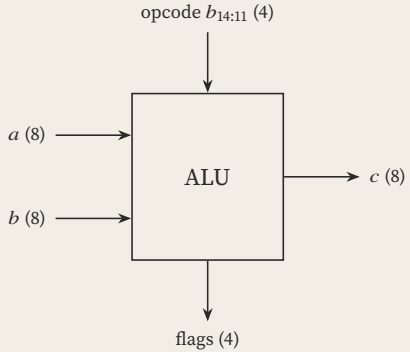
- `shiftl` doubles an unsigned number, as long as the leftmost bit was 0: `0000 0101` = 5 becomes `0000 1010` = 10.
- `shiftr` halves it, rounding down: `0000 1101` = 13 becomes `0000 0110` = 6.
- Twice `shiftl` multiplies by 4, three times by 8, and so on, just as adding a 0 on the right multiplies a decimal number by 10.

> **Where it breaks**
>
> `shiftl` loses the leftmost bit: `1000 0001` = 129 becomes `0000 0010` = 2. And `shiftr` brings in a 0, so a negative two's complement number turns positive: `1111 1100` = −4 becomes `0111 1110` = 126, not −2.

02

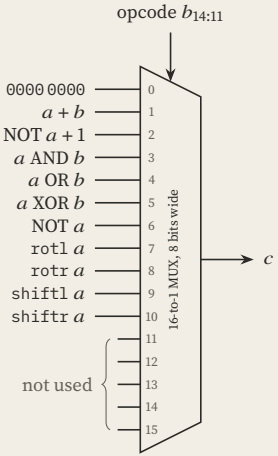
Putting the ALU together

The ALU from the outside



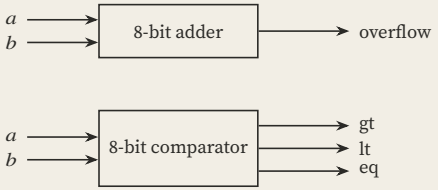
- Inputs: two bytes, a (from regA) and b (from regB), and the 4-bit opcode of the instruction.
- Outputs: a byte c (to regOUT) and four flags (to regFLAG).
- Inside: every unit of Lectures 16 and 17, the shifts and rotations of today, and one multiplexer.

Selecting the operation



- Every unit computes its result at the same time; the multiplexer `(Unit 7)` passes on the one the opcode selects.
- The inputs are in opcode order: input 1 is `add` (1 0001), input 10 is `shiftr` (1 1010). So the opcode is the select, with no decoding.
- `cmp` selects the constant 0: its answer is in the flags.

The flags



- Four flags, whatever the opcode: the adder's overflow `(Unit 16)` and the comparator's gt, lt and eq `(Unit 17)`.
- They go to regFLAG. After a `cmp`, the jumps `jmpgt`, `jmplt` and `jmpeq` test them.
- Remember: the comparator reads a and b as unsigned numbers.

One input, every answer

Opcode	Instr.	c
0000	cmp	0000 0000
0001	add	1110 1001
0010	neg	0110 1010
0011	and	0001 0010
0100	or	1101 0111
0101	xor	1100 0101
0110	not	0110 1001
0111	rotl	0010 1101
1000	rotr	0100 1011
1001	shiftl	0010 1100
1010	shiftr	0100 1011

- a = 1001 0110, b = 0101 0011. All eleven results exist inside the ALU at once; the opcode picks one.
- Flags: gt = 1, since 150 > 83 unsigned (although a is −106 in two's complement). Overflow = 0: −106 + 83 = −23 fits.
- Check a few rows by hand.

Why compute everything?

- The ALU does not wait for the opcode: all units work in parallel, and the multiplexer chooses. Simple to design, and the result is ready after the slowest unit plus the multiplexer.
- The price: gates that work for nothing most of the time.
- In a clocked design, the ALU sits between registers (regA and regB before, regOUT and regFLAG after): its longest path, through the adder, is part of the setup constraint (Unit 14).

Try it

1. a = `1100 0011`. What do `rotl`, `rotr`, `shiftl` and `shiftr` give?
2. Which ALU operation, used twice, multiplies an unsigned byte by 4? When does it fail?
3. Opcode 0101, a = `1111 0000`, b = `1010 1010`. What is c?
4. How many select bits does the multiplexer have, and how many of its inputs are not used?
5. Opcode 0001, a = `0111 1111`, b = `0000 0001`. What are c and the flags?

Try it

1. a = `1100 0011`. What do `rotl`, `rotr`, `shiftl` and `shiftr` give?
   `1000 0111`, `1110 0001`, `1000 0110`, `0110 0001`.
2. Which ALU operation, used twice, multiplies an unsigned byte by 4? When does it fail?
3. Opcode 0101, a = `1111 0000`, b = `1010 1010`. What is c?
4. How many select bits does the multiplexer have, and how many of its inputs are not used?
5. Opcode 0001, a = `0111 1111`, b = `0000 0001`. What are c and the flags?

Try it

1. a = `1100 0011`. What do `rotl`, `rotr`, `shiftl` and `shiftr` give?
   `1000 0111`, `1110 0001`, `1000 0110`, `0110 0001`.
2. Which ALU operation, used twice, multiplies an unsigned byte by 4? When does it fail?
   `shiftl`. It fails if one of the two leftmost bits is 1.
3. Opcode 0101, a = `1111 0000`, b = `1010 1010`. What is c?
4. How many select bits does the multiplexer have, and how many of its inputs are not used?
5. Opcode 0001, a = `0111 1111`, b = `0000 0001`. What are c and the flags?

Try it

1. a = `1100 0011`. What do `rotl`, `rotr`, `shiftl` and `shiftr` give?
   `1000 0111`, `1110 0001`, `1000 0110`, `0110 0001`.
2. Which ALU operation, used twice, multiplies an unsigned byte by 4? When does it fail?
   `shiftl`. It fails if one of the two leftmost bits is 1.
3. Opcode 0101, a = `1111 0000`, b = `1010 1010`. What is c?
   `xor`: `0101 1010`.
4. How many select bits does the multiplexer have, and how many of its inputs are not used?

5. Opcode 0001, a = `0111 1111`, b = `0000 0001`. What are c and the flags?

Try it

1. a = `1100 0011`. What do `rotl`, `rotr`, `shiftl` and `shiftr` give?
   `1000 0111`, `1110 0001`, `1000 0110`, `0110 0001`.
2. Which ALU operation, used twice, multiplies an unsigned byte by 4? When does it fail?
   `shiftl`. It fails if one of the two leftmost bits is 1.
3. Opcode 0101, a = `1111 0000`, b = `1010 1010`. What is c?
   `xor`: `0101 1010`.
4. How many select bits does the multiplexer have, and how many of its inputs are not used?
   4 select bits; inputs 11 to 15, so 5 are not used.
5. Opcode 0001, a = `0111 1111`, b = `0000 0001`. What are c and the flags?

Try it

1. a = `1100 0011`. What do `rotl`, `rotr`, `shiftl` and `shiftr` give?
   `1000 0111`, `1110 0001`, `1000 0110`, `0110 0001`.
2. Which ALU operation, used twice, multiplies an unsigned byte by 4? When does it fail?
   `shiftl`. It fails if one of the two leftmost bits is 1.
3. Opcode 0101, a = `1111 0000`, b = `1010 1010`. What is c?
   `xor`: `0101 1010`.
4. How many select bits does the multiplexer have, and how many of its inputs are not used?
   4 select bits; inputs 11 to 15, so 5 are not used.
5. Opcode 0001, a = `0111 1111`, b = `0000 0001`. What are c and the flags?
   c = `1000 0000`; overflow = 1 (127 + 1 does not fit), gt = 1, lt = eq = 0.

03

Summary

Summary

- Rotations move every bit one place and wrap the end bit around; shifts lose it and bring in a 0. Both are only wires.
- `shiftl` doubles an unsigned number and `shiftr` halves it, while no 1 falls off.
- The ALU computes every operation at once; a 16-to-1 multiplexer, selected by the opcode $b_{14:11}$, picks the result c.
- Four flags come out whatever the opcode: overflow, gt, lt and eq.
- `cmp` gives $c = 0$; its answer is in the flags.

Before the next lecture

- Read Section 6.4 of the book.
- In Logisim-evolution, build the 8-bit ALU from your adder, comparator and logic units, with splitters for the shifts and rotations and a 16-to-1 multiplexer. Check the table of this lecture.
- Read Section 6.5 of the book.

> **Next lecture** (Unit 19)
>
> Memory: registers, addressing, byte-addressable RAM and the register file.